



S.G. Ganesh

Understanding Increment and Decrement Operators in C

Almost all programmers who work in C-based languages use the ++ and -- operators. Let's look at some interesting aspects of these operators in C.

Increment/decrement operators are good examples to explain the curt (concise) syntax of C language:

```
void strcpy(char *t, const char *s) {
    while(*t++ = *s++);
}
```

In this *strcpy* function, the *while* loop continues copying *chars* from source to target, till it reaches the end of the string (`'\0'` character).

Here is another example; can you guess the output of this code segment?

```
char str[] = "oh", *p = str;
while (*p)
    ++*p++;
printf(str);
```

How is the expression `++*p++` treated? The postfix increment/decrement is part of the operators with highest precedence; the dereference and prefix increment/decrement is at the next precedence level and their associativity is from right to left. So, the expression is treated as: `((*(p++))++)`. *p* is a pointer to string "oh". In the *while* loop, *p++* is executed first, but its effect doesn't take place till the statement execution completes. The result of *p++* (which is still the value of *p*) is dereferenced with `*`. For the first character it is 'o'. Now, prefix ++ increments the character value, so its value is 'p'. The same happens for the next character, 'h', which becomes 'i'. Now the end of the character is reached, so the *while* loop terminates. The *printf* now prints the modified string, which is "pi".

We can use `++/--` operators to illustrate the 'maximum munch' rule in C lexical analysis. Simply stated, the lexical analyser (lexer) looks for a match for the longest token. For example, consider the expression *a+b*, where *a* and *b* are integer variables. Given this as input in a C program, the lexer might give VARIABLE

(*a*), PLUS (+), VARIABLE(*B*) as output, which is a valid sequence of tokens for compilation to succeed.

Now, how about the expression *a++b*? It is treated as *a + b*, and not *(a + (b))*, and results in a compiler error. How about *a+++b*? It becomes *((a++) + b)*, so it's fine. Expression *a++++b*? It is *((a++) ((a++) + b))*, so it is a compiler error. Hmm, how about *a+++++b*? It is *((a++) ++)((a++) + b)*, which is again a compiler error. Note that, in all these cases, if we use explicit white-space or parenthesis, we can get valid expressions: *(a + (b))*, *((a++) + b)*, *((a++) + (b))*, and *((a++) + ((a++) + b))*. If we try adding more *+s* after this, we cannot get a valid expression for integer arguments.

Today, C-based languages (C++, Java, etc) rule the programming world; because of this, almost all programmers are aware of the ++ and -- operators, and how powerful they are in expressing operations. What even experienced programmers do not know is that the C family was the first to introduce the ++ and -- operators! Ken Thompson, while implementing the B compiler, noticed that increment operations generate more compact code than adding 1 to a variable and storing it back. In other words, an increment was one instruction while adding 1 and storing it back was more than one instruction.

So he implemented the prefix ++ (and --) operators, and generalised it by adding a postfix version. Interestingly, this short code-generation advantage is still true for recent languages. For example, in Java, integer increment would generate one instruction (*iinc*) compared to the four instructions for adding 1 to the variable and storing it back (*iload_x, iconst_1, iadd* and *istore_x* instructions)! 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is 'Cracking the C, C++ and Java Interview', published by Tata McGraw-Hill. You can reach him at ssganesh@gmail.com.